

Datatypes – Simplified Notes

1. What is a datatype?

A **datatype** tells us:

1. What values a data item can store.
2. What operations can be performed on those values.
3. What values are invalid or meaningless.

Example

A student's marks can be stored as an integer between 0 and 100.

- Valid values: 45, 78, 100.
- Invalid values: -5, 105.
- Meaningful operations: addition, subtraction, comparison.
- Meaningless operations: multiplying one mark by another.

Thus, a datatype helps the computer detect incorrect data and invalid operations.

2. Why do we need datatypes?

Real-world data may contain errors:

- A field name may be misspelled.
- A mark may be entered as 105 instead of 100.
- A total bill may not match the sum of its items.
- A date may be written in an invalid form.
- A name may contain inappropriate symbols.

If every program checked all these errors manually, the algorithm would become complicated and difficult to understand.

Datatypes solve this problem by placing restrictions on data. The computer can then identify incorrect values automatically.

Example

If gender is defined as a datatype that allows only M or F, then values such as X or Male123 can be rejected.

3. Sanity checking in datasets

Sanity of data means checking whether the data is reasonable, valid, and consistent.

3.1 Classroom dataset

Important fields include:

Field	Meaning	Valid values or rules
Marks	Subject marks	From 0 to 100
Total marks	Sum of three subjects	From 0 to 300
Name	Student name	Alphabetic characters, spaces, full stops, or hyphens
City	Town or city name	Must be a recognised city
Gender	Student gender	M or F
Date of birth	Month and day	Valid month and date
Total marks	Derived value	Must equal Maths + Physics + Chemistry

For example:

$\text{totalMarks} = \text{mathsMarks} + \text{physicsMarks} + \text{chemistryMarks}$

Under Revision

If a student has marks 65, 70, and 80, the total must be:

$65+70+80=215$
 $65+70+80=215$

The total field is a **redundant field** because it can be calculated from the subject marks. However, it is useful for detecting data-entry mistakes.

3.2 Shopping-bill dataset

Important rules include:

- Quantity, price, and amount may be fractional.
- The cost of an item should be:
 $\text{cost} = \text{quantity} \times \text{price}$
- The total bill should equal the sum of all item costs.
- Customer names, shop names, and item categories must follow valid naming rules.

3.3 Words dataset

Important fields include:

- The word.
- Its part of speech.
- Its letter count.

The letter count must match the number of alphabetic letters in the word.

For example:

- Monday has 6 letters.
- morning. has 7 letters; the full stop is not counted.

4. Basic datatypes

The chapter introduces three basic datatypes:

1. Boolean.
2. Character.
3. Integer.

4.1 Boolean

A **Boolean** value has only two possible values:

- True
- False

Boolean values are used for conditions and decisions.

Examples

- $\text{marks} > 50$
- $\text{gender} == \text{"F"}$
- $\text{city} == \text{"Chennai"}$

Boolean operations

Operation	Meaning	Number of inputs	Result
AND	True only when both conditions are true	2	Boolean
OR	True when at least one condition is true	2	Boolean
NOT	Reverses the value	1	Boolean

Truth table

A	B	A AND B	A OR B
True	True	True	True
True	False	False	True
False	True	False	True

Under Revision

A	B	A AND B	A OR B
False	False	False	False

For example:

`age >= 18 AND citizen == True`

This is true only if both conditions are true.

4.2 Character

A **character** is one individual symbol.

Characters may include:

Alphabetic characters

A, B, C, ..., Z

a, b, c, ..., z

Digits

0, 1, 2, ..., 9

Special characters

space, comma, full stop, hyphen, underscore, slash, question mark

A character is different from a string. For example:

- M is one character.
- Male is a string containing four characters.

Main operation

The most useful operation on characters is equality checking.

`gender == 'M'`

The result is Boolean:

True or False

4.3 Integer

An **integer** is a whole number.

Examples:

..., -3, -2, -1, 0, 1, 2, 3, ...

Integers may be:

- Negative.
- Zero.
- Positive.

Operations on integers

Operation	Result
Addition	Integer
Subtraction	Integer
Multiplication	Integer
Division or quotient	Integer, depending on the definition
Equality comparison	Boolean
Less-than comparison	Boolean
Greater-than comparison	Boolean

Examples:

`5 + 3 = 8`

Under Revision
10 - 4 = 6
6 * 2 = 12
7 > 3 = True

5. Compound datatypes

A **compound datatype** is created by combining simpler datatypes.

The main compound datatypes are:

1. Strings.
2. Lists.
3. Records.

5.1 Strings

A **string** is a sequence of characters.

Examples:

"Avinash"

"Chennai"

"Computational Thinking"

"ABC123"

A string can contain letters, numbers, spaces, and special characters.

String operations

Operation	Meaning	Result
Append	Join two strings	String
Equality	Check whether two strings are equal	Boolean
StartsWith	Check beginning of a string	Boolean
EndsWith	Check ending of a string	Boolean
Contains	Check whether one string occurs inside another	Boolean

Examples

"Hello" append " World"

Result:

"Hello World"

"Chennai" startsWith "Che"

Result:

True

"morning." endsWith "."

Result:

True

5.2 Lists

Under Revision

A **list** is an ordered collection of elements.

Example:

[46, 88, 92, 55, 88, 75]

Important features of a list:

- Elements have a position.
- The same value can occur more than once.
- A list can contain zero or more elements.
- Usually, all elements have the same datatype.

List operations

Operation	Meaning
Append	Add an element or list at the end
Add	Add an element at the beginning
Equality	Check whether two lists are equal
IsEmpty	Check whether the list has no elements
Head	Return the first element
Tail	Return the list without its first element

Example

Suppose:

marks = [46, 88, 92]

Appending 75 gives:

[46, 88, 92, 75]

The head is:

46

The tail is:

[88, 92]

The head and tail operations are valid only when the list is not empty.

5.3 Records

A **record** is a collection of named fields.

Each field has:

- A name.
- A value.
- A datatype.

Example

```
Student = {  
    name: "Ritika",  
    gender: "F",  
    mathsMarks: 87,  
    physicsMarks: 64  
}
```

A field can be accessed using its name.

Student.name

Under Revision

Result:
"Ritika"

Student.mathsMarks
Result:
87

A record is suitable for representing a student, shopping bill, word, train, or rectangle.

6. Subtypes

A **subtype** is a restricted version of an existing datatype.

A subtype restricts:

- The values that may be stored.
- The operations that may be performed.
- Additional rules that the value must satisfy.

6.1 Character subtypes

The Gender datatype is a subtype of Character.

Allowed values:

'M', 'F'

Although a character datatype can store many symbols, the gender subtype allows only these two values.

6.2 Integer subtypes

Sequence number

A sequence number may allow only values from 0 to a maximum value.

SeqNo = 0 to MAX

Negative values are not allowed.

The main meaningful operation may be equality checking.

Marks

Subject marks:

Marks = 0 to 100

Total marks:

TMarks = 0 to 300

Allowed operations include:

- Addition.
- Subtraction.
- Equality comparison.
- Less-than comparison.
- Greater-than comparison.

Multiplying two marks is not normally meaningful.

Count

A count is a non-negative integer.

Count = 0 to MAX

A count can be added, subtracted, and compared.

Summary table

Subtype	Allowed values	Typical operations
SeqNo	0 to MAX	Equality
Marks	0 to 100	Addition, subtraction, comparison
TMarks	0 to 300	Addition, subtraction, comparison
Count	0 to MAX	Addition, subtraction, comparison

6.3 String subtypes**Names**

A name may contain:

- Alphabetic characters.
- Spaces.
- Full stops.

Example:

"Boina Avinash Kumar"

City

A city is similar to a name but should normally be selected from a recognised list of cities.

Example:

"Chennai"

"Madurai"

"Vellore"

Words

A word may contain letters, digits, or special characters.

Examples:

"Monday"

"building;"

"fire-eyed"

Word category

A word category must come from a fixed list.

Examples:

Noun

Pronoun

Verb

Adjective

Adverb

Only equality checking is generally needed.

Under Revision

Item name and item category

Item names and categories should also come from approved sets.

Examples:

Item name: Carrots, Soap, Socks

Item category: Vegetables, Toiletries, Footwear

7. Transforming data

Some values are difficult to store or process directly. We can transform them into another form.

After transformation, we can:

- Store them easily.
- Compare them.
- Perform calculations.
- Convert them back for display.

The chapter discusses dates and fractional values.

7.1 Representing dates

A date such as:

6 Dec

looks like a string. However, comparing strings may not correctly tell us which date comes earlier.

For example, it is useful to determine whether:

6 Dec > 10 Jan

To make date operations easier, the date can be converted into the number of days since 1 January.

The chapter uses:

1 January = day 0

Therefore:

- 6 January = 5
- 31 January = 30
- 1 February = 31
- 1 March = 59 in a non-leap year
- 6 December = 339
- 31 December = 364

Now dates can be compared as integers.

If:

$D1 < D2$

then date D1 occurs before date D2.

The gap between two dates can be calculated by subtraction.

Date conversion operations

Operation	Purpose
store(date)	Convert a date string into its number
print(number)	Convert the number back into a date string
Equality	Check whether two dates are equal
Less than	Check which date comes earlier
Gap	Find the number of days between dates

Important restriction

Dates should not be multiplied or divided, even though they are represented internally as integers.

7.2 Representing fractional values

Shopping data may contain fractional values such as:

62.75

1.5

0.5

The book suggests multiplying values by 100 and storing them as integers.

Example

$62.75 \times 100 = 6275$

So:

$62.75 \rightarrow 6275$

This is suitable when values have at most two decimal places.

Conversion operations

`store(62.75) → 6275`

`print(6275) → 62.75`

This transformation allows addition, subtraction, and comparison using integers.

Important limitation

Multiplication and division are not directly suitable after this transformation.

For example, if both values are multiplied by 100, their product is multiplied by 10,000. Therefore, the result must be adjusted carefully.

8. Datatypes for complete datasets

A complete data element is usually represented as a record. A dataset containing many such elements is represented as a list of records.

8.1 Student datatype

A student record may contain:

Field	Datatype
uniqueId	SeqNo
studentName	Names
gender	Gender
dateOfBirth	Date
townCity	City
mathsMarks	Marks
physicsMarks	Marks
chemistryMarks	Marks
totalMarks	TMarks

The complete classroom dataset is:

List of StudentMarks

8.2 Word datatype

A word record may contain:

Field	Datatype
uniqueId	SeqNo
word	Words
partOfSpeech	WordCategory
letterCount	Count

The complete words dataset is:

List of ParagraphWord

8.3 Shopping-bill datatype

A purchased item can be represented as:

Field	Datatype
itemName	Items
category	ItemCategory
quantity	Quantity
price	Price
cost	Cost

A shopping bill can be represented as:

Field	Datatype
uniqueId	SeqNo
storeName	Names
customerName	Names
items	List of BillItem
billAmount	Amount

The complete shopping-bill dataset is:

List of ShoppingBill

This is a **list containing records, where each record contains another list.**

9. Main idea of the chapter

The central idea of Chapter 5 is:

A datatype defines the valid values and valid operations for data.

The chapter progresses from simple to complex:

Boolean, Character, Integer

Under Revision



String, List, Record



Subtypes



Transformed datatypes



Complete dataset datatypes

Simple example

Marks

is more useful than simply saying:

Integer

because Marks tells us that:

- The value must be between 0 and 100.
- Marks can be compared.
- Marks can be added.
- Multiplying two marks is not meaningful.

Therefore, datatypes improve:

- Data accuracy.
- Program safety.
- Algorithm clarity.
- Error detection.
- Communication between programmers.

Quick revision points

1. A datatype specifies valid values and valid operations.
2. Boolean has only True and False.
3. Character stores one symbol.
4. Integer stores whole numbers.
5. String is a sequence of characters.
6. List is an ordered collection of elements.
7. Record is a collection of named fields.
8. A subtype is a restricted datatype.
9. Dates can be transformed into day numbers.
10. Fractional values can be scaled and stored as integers.
11. A dataset is commonly represented as a list of records.
12. Datatypes help prevent invalid data and meaningless operations.